

# Salareen / AOEP

## Platform Architecture

Agentic Online Education Platform — version 0.19.22 · 2026-07-14

Frontend, backend, accounts, data flow, network traffic, live classes, and deployment. Diagrams are generated from code by `scripts/build_architecture_pdf.py` and stay in sync with the system.

# Overview

Salareen (referred to as "AI Classroom" in parts of the codebase) is a multi-service education platform for live AI-taught classes, mobile Drive-Mode audio lessons, adaptive learning, language learning, rewards, careers-to-skills matching, compliance controls, and third-party integrations. The **same codebase** runs local, cloud, or edge by environment configuration only.

## Shape of the system

- **Monorepo.** Backend = Python/FastAPI microservices; web = Next.js (apps/web); mobile = Expo React Native (apps/mobile).
- **Shared library.** packages/shared (aoep\_shared) holds all cross-service logic and a provider abstraction (LLM, Speech, Vision, Media/LiveKit, Payment, Object Store, Search). Implementations are chosen by env (DEPLOY\_MODE = local | cloud | edge, plus per-component <COMPONENT>\_MODE) — no code forks between local and cloud.
- **Offline-first.** Heavy providers target real endpoints; without them the code degrades gracefully (RAG-grounded tutor fallback, device TTS, in-memory stores) so the teaching loop still runs.

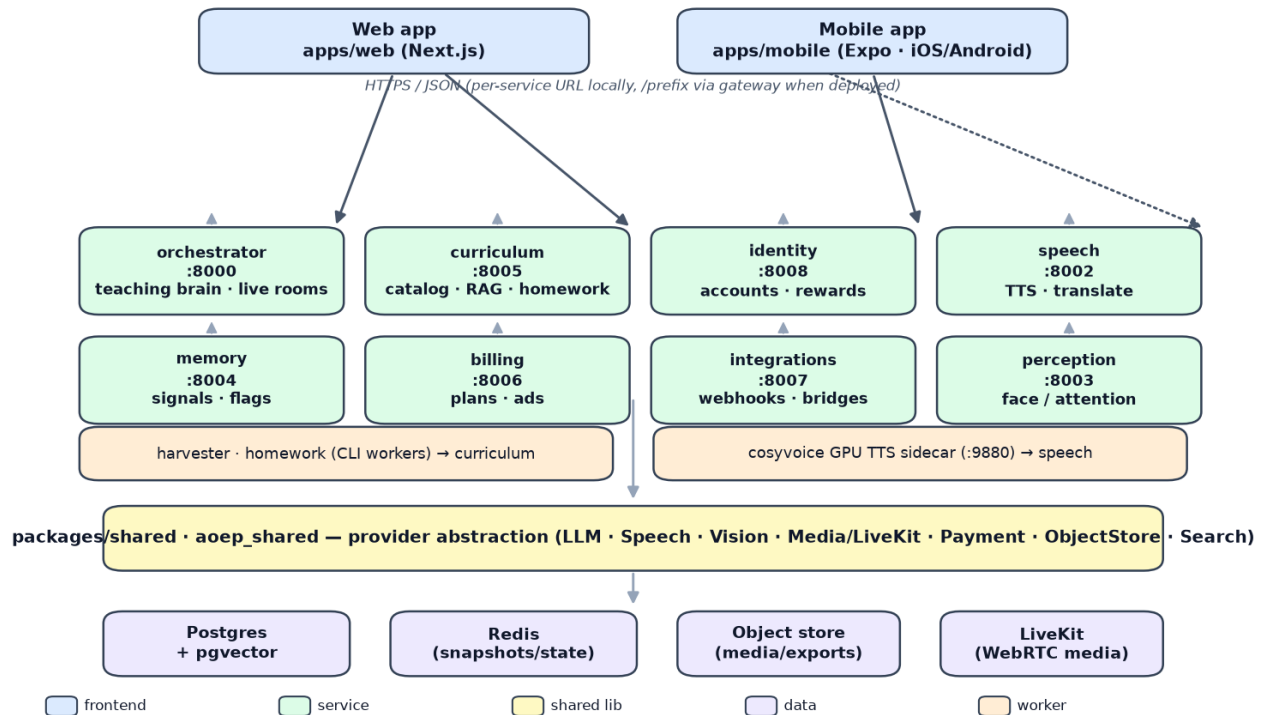
## Services and local ports

| Service      | Pkg          | Port | Owns                                                                        |
|--------------|--------------|------|-----------------------------------------------------------------------------|
| orchestrator | orchestrator | 8000 | teaching brain: sessions, tutor, live rooms, group classes, assessment, HIL |
| speech       | speech_gw    | 8002 | TTS routing/synthesis, translation, language learning                       |
| perception   | perception   | 8003 | consent-gated face recognition / engagement (OpenCV YuNet+SFace)            |
| memory       | memory       | 8004 | consent, mastery/behavior signals, feature flags, surveys, mascots          |
| curriculum   | curriculum   | 8005 | catalog, RAG corpus, decks, audio courses, jobs, homework, notifications    |
| billing      | billing      | 8006 | plans, entitlements, payment methods, checkout, ads                         |
| integrations | integrations | 8007 | webhooks, LMS, finance, cloud connectors, Zoom/Teams/Meet bridges           |
| identity     | identity     | 8008 | accounts, auth, students, rewards, enrollments, preferred language          |
| harvester    | harvester    | —    | CLI worker: crawl/generate/export courses -> curriculum                     |
| homework     | homework     | —    | CLI worker: generate / OCR / authorship / autograde                         |
| cosyvoice    | (server.py)  | 9880 | GPU CosyVoice 2 TTS sidecar (POST /tts) for clone narration                 |

Every HTTP service also exposes /health, /version, /\_\_meta, /metrics, and /telemetry/\* via create\_service(). In Docker/k8s every service listens on :8000 internally; the :800x ports above are local dev only.

# System topology

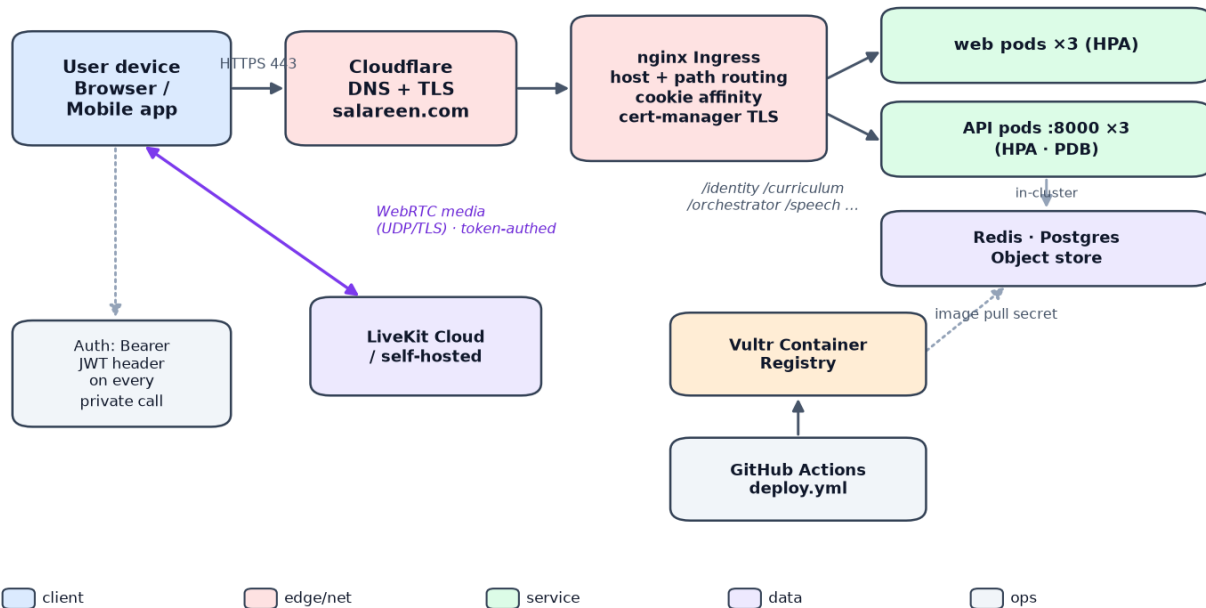
## 1 · System topology — frontend, backend, shared, data



Clients (web + mobile) call the FastAPI services over HTTPS/JSON. Locally each service has its own port; when deployed the browser reaches them through same-origin path prefixes (/identity, /curriculum, ...) that the gateway rewrites. Every service depends on aoep\_shared, which in turn talks to the data tier. CLI workers (harvester, homework) feed the curriculum service; the cosyvoice GPU sidecar serves the speech gateway.

# Network traffic flow

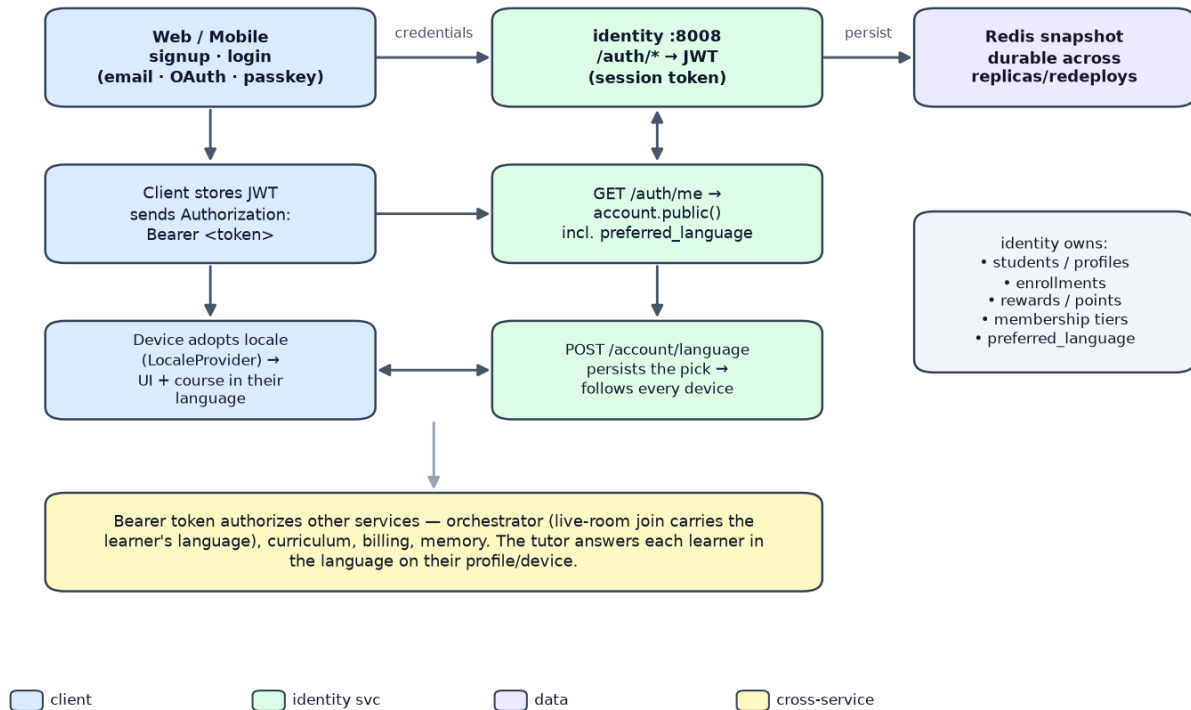
## 2 · Network traffic flow — device → edge → services → data



User traffic enters via Cloudflare (DNS + TLS) to the nginx Ingress, which does host + path routing with cookie session affinity and cert-manager TLS, then forwards to the web pods or the API pods (all on :8000). Services reach Redis / Postgres / object storage in-cluster. Real-time class media is a separate path: clients connect directly to LiveKit (Cloud or self-hosted) over token-authenticated WebRTC. Images are pulled from the Vultr Container Registry using an image-pull secret; GitHub Actions builds and rolls them. Every private HTTP call carries an `Authorization: Bearer <JWT>` header.

# Accounts, auth & language

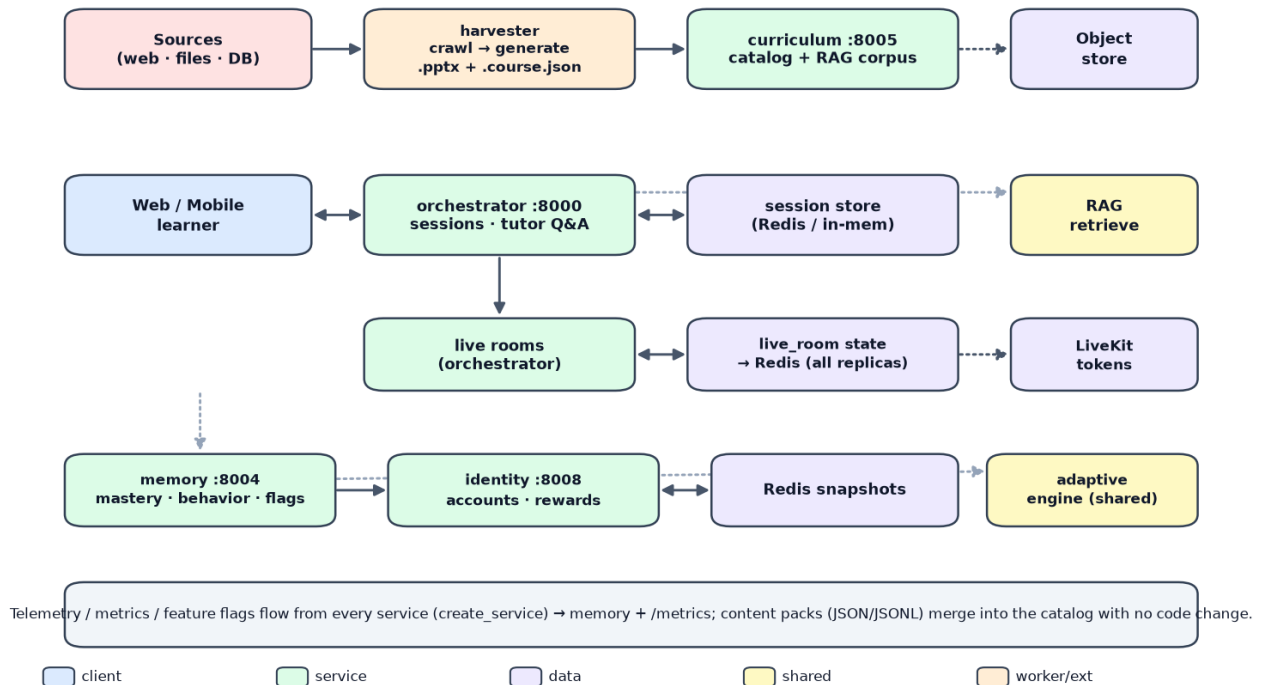
## 3 · Accounts, auth & language — identity of the learner



Signup/login (email, OAuth, or passkey) mints a JWT from the identity service; the client sends it as a Bearer token. GET /auth/me returns the account, including preferred\_language. The device adopts that locale (LocaleProvider) so the UI and course appear in the learner's language, and a change via the picker persists with POST /account/language so it follows the learner to every device. Identity is a system of record backed by a Redis snapshot (durable across replicas and redeploys) and owns students, enrollments, rewards/points, and membership tiers. The Bearer token authorizes the other services, and the tutor answers each learner in the language on their profile/device.

# Data flow & stores

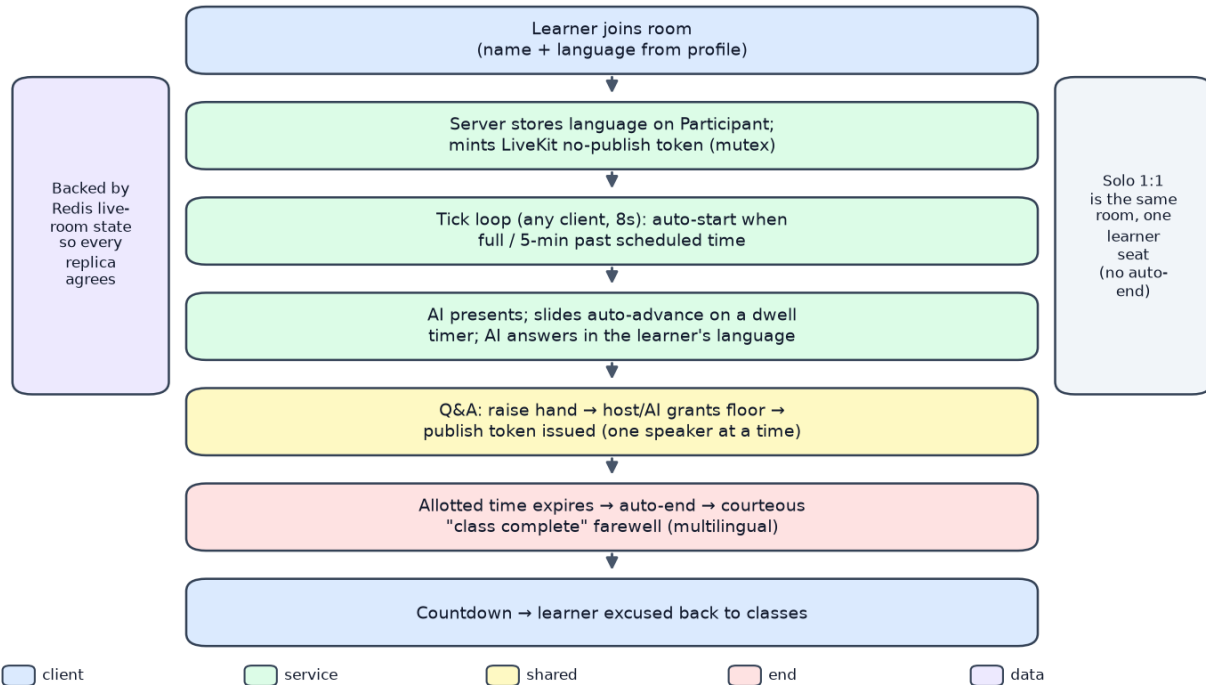
## 4 · Data flow & stores — content, sessions, signals, state



Content: the harvester crawls/generates courses (exporting `.pptx` + `.course.json`) into the curriculum catalog and RAG corpus. Teaching: apps drive orchestrator sessions (session store, Redis in cloud) that retrieve grounding passages via RAG and answer. Live rooms keep their state in Redis so every orchestrator replica serves the same room, and mint LiveKit tokens. Learner signals (mastery/behavior/flags) live in memory and feed the adaptive engine; accounts and rewards live in identity. Telemetry, metrics, and feature flags flow from every service; content packs merge into the catalog with no code change.

# Live-class lifecycle

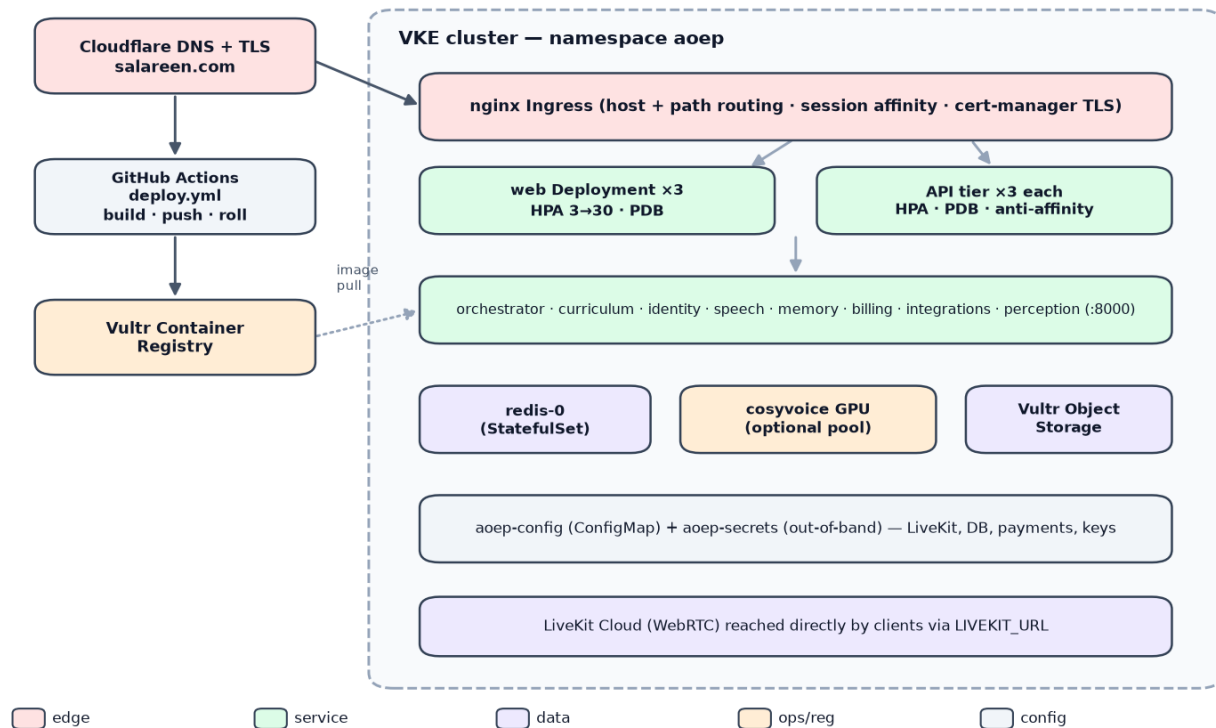
## 5 · Live-class lifecycle — join → teach → Q&A → complete



A learner joins with a name and their language; the server records the language on the Participant and mints a no-publish LiveKit token (the single-speaker mutex). A tick loop (any client, every 8s) auto-starts the class when the room is full or 5 minutes past the scheduled time, auto-advances slides on a dwell timer, and auto-ends when the allotted time expires — showing a courteous multilingual "class complete" farewell before excusing everyone. Q&A uses a raise-hand queue; the host/AI grants the floor, which issues a publish token so exactly one person talks. A solo 1:1 class is the same room scaled to a single learner seat (open-ended, no auto-end).

# Deployment (Vultr VKE)

## 6 · Deployment — Vultr Kubernetes Engine (namespace aoep)



GitHub Actions (`deploy.yml`) builds images, pushes them to the Vultr Container Registry, and rolls the cluster. In the `aoep` namespace, the `nginx Ingress` fronts a `web Deployment` (HPA 3→30, PDB) and the `API tier` (each service ×3 with HPA, PDB, and zone anti-affinity, all on `:8000`). Redis runs as a `StatefulSet`; an optional GPU pool runs `cosyvoice`; media/exports use Vultr Object Storage. Configuration is split between `aoep-config` (ConfigMap) and `aoep-secrets` (managed out-of-band so `apply -k` never clobbers real secret values). Clients reach LiveKit directly via `LIVEKIT_URL`.

# Rebuild & references

Regenerate this document (diagrams + PDF) after any architecture change:

```
pip install matplotlib==3.11.0 reportlab==5.0.0
python3 scripts/build_architecture_pdf.py
```

Deeper references: docs/architecture.txt (canonical text), docs/api-reference.txt (every endpoint), docs/hosting.txt / docs/scalability.txt, the per-service README.txt, and infra/k8s-vke/RUNBOOK.txt.